

Web Take Home Exercise

Build two web apps that talk to each other across an iframe boundary using `postMessage`.

- 1 Data App:** owns all Imgur API access. It has no UI. It answers data requests that arrive from the parent window.
- 2 Host App:** embeds the Data App in a hidden iframe and owns the entire user interface. It requests data over `postMessage` and renders what comes back.

Why we ask for this: Our product surface runs inside an iframe hosted by a partner's website, the embedded app owns the API access and the credentials, and the host app asks for what it needs through a typed `postMessage` contract and awaits a typed answer. The interesting part of this exercise is that boundary, not the album list.

App 1: Data App (the iframed app)

- No UI. It renders nothing a user sees, so the iframe that hosts it can be hidden or zero sized.
- Owns all Imgur access. The Client ID and every `fetch` call live here and nowhere else.
- Listens for data requests from the parent window and answers each one with a typed response. At minimum it should support searching albums for a query, and returning the images for a given album.
- Reports failures back across the boundary as typed error responses. A failed Imgur call should never leave the parent waiting forever.

App 2: Host App (the parent)

- A separate app, served on a different port so the two apps have different origins.
- Embeds the Data App in a hidden iframe.
- Owns the entire UI: a text entry component to query albums, the list of albums returned for that query, and a gallery of images for the selected album.
- Must not call the Imgur API itself, and must never hold the Client ID.
- Drives everything by request. The user types a query, the Host App asks the Data App for matching albums, and renders the response. Selecting an album works the same way.
- Loading and error states are derived from the state of the in flight request.

Messaging requirements

- Define a typed message contract shared by both apps, covering both the requests and the responses. Both sides should agree on the same types rather than each guessing at the shape.
- Handle the failure paths, not just the happy one. A request that errors, or that never gets an answer at all, needs to surface to the user rather than hang.

Requirements and preferences

- Host App UI built in React.
- Use `async / await` for API calls.
- Prefer TanStack Query and TanStack Router in the Host App.
- You can use any third party libraries you deem appropriate, but include a brief explanation of why you chose them.
- You are free to implement any design or architecture you think will help you solve the problem.
- Write code with the same quality you would on a normal project, and leave comments as you deem appropriate.
- Both apps need to be runnable locally with documented commands. A monorepo or two separate projects are both fine.

Imgur API

Use the gallery search endpoint documented here:

<https://apidocs.imgur.com/#3c981acf-47aa-488f-b068-269f65aee3ce>

You do not need to create an account. Use the Client ID below.

```
curl --location --request GET \  
  "https://api.imgur.com/3/gallery/search?q_all=cats&q_type=album" \  
  --header "Authorization: Client-ID 2d086962f60c89e"
```

Imgur API Client ID: 2d086962f60c89e

This Client ID is a throwaway for the exercise, so calling Imgur straight from the browser is fine here. In a real product a credential like this belongs server side behind our own endpoint. We are happy to talk about that tradeoff in the follow up conversation.

Bonus points

None of these are required. Pick up whatever you find interesting.

- **Talk about the SDK layer.** In production, host developers do not hand roll the iframe, the listener, and the request correlation. They install a client SDK that turns all of it into typed method calls that return promises. In your README, describe what that SDK should own and why: the public API it would expose, how it hides iframe lifecycle and origin checks, how the message contract stays versioned and backwards compatible as the embedded app ships new capabilities, and how errors surface to the host developer. Sketching or implementing a thin version of that client is welcome but not expected.
- Caching or request deduplication inside the Data App, so repeated queries do not hit Imgur again.
- Cancelling or discarding requests that have been superseded, and timing out requests that never get an answer.
- Tests around the messaging layer.
- Selecting an image from the gallery to display it fullscreen, with the ability to refresh and see the same image.
- Thoughtful loading, empty, and error states.

Submitting the exercise

- Create a git repo for the exercise and commit like you would on a normal project.
- Add a README with a brief overview of the project, how to run both apps, and the choices you made.
- Share the URL of the repo with us, or zip it up and send it to sam@playlucra.com.

Please feel free to reach out if you have any questions.